

TRABAJO PRACTICO Tema Algoritmo CYK

Teoría de la Computación
Trabajo Práctico 2do semestre de 2025

Consideraciones generales

Implementar los siguientes ejercicios usando una base de datos postgres y programando en pl/pgsql. La entrega debe realizarse el día martes __ de noviembre. Debera incluir 3 tests unitarios con los cuales mostrar la funcionalidad. La entrega consta de una demo presencial durante la clase de ese día, y el envío por mail del código hasta las 18hs de ese día. No se recibirán TPs luego de ese momento. El trabajo práctico puede realizarse en grupos conformados por 2, 3 o 4 personas. No es necesario entregar informe, ni decisiones tomadas, ni conclusiones. Un diseño adecuado del código es parte de la evaluación.

Enunciado

Programar el algoritmo de parsing CYK usando funciones en pl/pgsql de postgres. Este TP se realiza con fines didacticos, para comprobar en forma practica el ciclo completo de un parser, desde el diseño de una gramática libre de contexto hasta su implementación mediante un parser.

Los strings a reconocer son expresiones json sencillas que respondan al siguiente formato

```
{“a”:10}
{“a”:10,”b”:’hola’}
{“a”:’hola’,”b”:’chau’,”c”:’}
{“a”:10,”b”:’hola’,”c”:{“d”:’chau’,”e”:99},”f”:{}}
{}
{“a”:10,”b”:’hola’,”c”:{“d”:’chau’,”e”:99,”g”:{“h”:12}},”f”:{}}
```

Los elementos válidos dentro del json pueden ser:

- Números enteros,
- Strings con letras (no es necesario incluir numeros en los strings) y blancos
- Elementos json (sin limite de anidamiento)

La parte 1 del TP consiste en diseñar una gramática GLC que reconozca las expresiones json válidas. Esta parte deberá tener un documento de texto que incluya como mínimo dos ejemplos de expresiones válidas con su árbol de parsing asociado (el arbol usado para reconocerlas).

La parte2 del TP consiste en aplicar todos los algoritmos que cubrimos en la materia (algoritmos de limpieza y pasaje a FNC). Cada algoritmo debe presentarse en un documento de texto indicando GLC de input, GLC de output y mencionar los pasos intermedios más importante de cada algoritmo. Por ejemplo el conjunto de símbolos NO alcanzables, o el conjunto de simbolos NO generadores. No es necesario mostrar el paso a paso de cada algoritmo, solo los puntos más importantes. Esta parte del TP como conclusión final debe tener la gramática en FNC generada (es la que vamos a cargar en la base de datos). Se valorará que con esta nueva gramática vuelvan a realizar los arboles de parsing de los mismos strings json de los ejemplos usados en el paso1.

Nota: Hasta aca, no es necesario programar nada, las partes 1 y 2 del TP se presentan en un documento de texto.

La parte 3 del TP consiste en implementar en una base de datos postgresQL la lógica del algoritmo CYK mediante el uso de funciones en plpgsql y de tablas de la base de datos y de SQL. Para esta parte vamos a definir algunas tablas que todos los grupos deberán utilizar, el resto del diseño de la solución queda a criterio de cada grupo, sin embargo las tablas indicadas más abajo en el enunciado deben usarse sin cambios, para facilitar la corrección del TP.

Tabla GLC_en_FNC
start boolean,
parte_izq text,
parte_der1 text,
parte_der2 text,
tipo_produccion smallint (1 indica $Var \rightarrow terminal$, 2 indica $Var \rightarrow Var1Var2$)

Esta tabla deberá utilizarse tal como está definida (le pueden agregar índices si los requieren) y deberá cargarse mediante un script SQL que tendrá un comando INSERT por cada producción en la gramática FNC generada en la parte 2 del TP. Atencion: para probar el parte 3, no es necesario hacerlo con la gramatica generada en los puntos 1 y 2. Mas aun, el punto 3 se puede probar con cualquier gramatica, mientras la misma este en FNC, se recomienda usar una gramatica sencilla, como la usada en el ejemplo que se vio en clase.

Tabla matriz_cyk
i smallint,
j smallint,
x[] text

x[] es un vector de text, vamos a usar vectores para representar cada conjunto de variables X_{ij} de la matriz. Los elementos de los arreglos X_{ij} serán variables de la GLC. Setear el contenido de estos arreglos es una parte importante de esta parte del TP, es decir programar la lógica de CYK usando funciones de postgresQL.

Deberán usar una función llamada setear_matriz(fila int) que será llamada de la siguiente manera por otra función de mayor nivel.

setear_matriz(1) para setear la primera fila de la matriz triangular (cargar los X_{ii})
setear_matriz(2) para setear el segundo nivel de la matriz triangular (carga los $X_{i,i+1}$)
setear_matriz(3) para setear el nivel 3 de la matriz triangular (por ejemplo el $X_{2,4}$)
Y así sucesivamente hasta
setear_matriz(N) donde N es la longitud del string que representa la expresion json

La expresion json, se debera almacenar en algun lugar (tabla quizas o alguna variable), ya que la debera acceder la funcion setear_matriz.

La funcion de mayor nivel podria ser algo tipo cyk(string) que retorne true o false, de manera que se podria invocar de la forma:

```
select cyk('{"a":10,"b":\'hola\', "c":{"d":\'chau\', "e":99}, "f":{}}');
```

Parte 4:

- Deberán proveer, un query en SQL para mostrar la GLC cargada en la tabla GLC_en_FNC.
- Deberán proveer un query en SQL que muestra la matriz triangular de alguna forma visual, similar a la que usamos en clase (o sea abajo la fila de los X_{ij})

Parte 5: Armar una GLC en FNC nueva (ready for CYK) que reconozca algún tipo de expresión. Puede ser alguna expresión aritmética, o la que se les ocurra. Luego pueden crear un script SQL para insertar en la tabla GLC_en_FNC la nueva GLC (borrando la anterior GLC). (Ojo con perder los datos de la tabla GLC_en_FNC). Luego podrían probar el algoritmo con algún string válido para la nueva gramática.